Canopy clustering is a bit trickier to re-design as a distributed algorithm, because it has only one step: drawing canopy centroids from the dataset and filtering out points within a certain distance from them so that they won't later be selected as centroids). The issue is that in this step, for each centroid drawn from the dataset, we need to go through the whole dataset for the filtering part – in theory for each centroid we would just need to process those points in its canopy but… we can't identify them in advance!

To get to a good solution, it can be useful to think about what's the real goal for canopy clustering: we want to get a set of canopy centroids that are no closer to each other than some distance T_2. The key point is that between any two of these canopies centers the distance **must** be above a minimum value, so that the canopies won't overlap too much; as we mentioned, this distance is similar to the "core distance" in DBSCAN, and points within a radius T_2 can be assumed to belong to the same cluster with high probability.